

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

DEPARTAMENTO DE INGENIERÍA EN SISTEMAS COMPUTACIONALES



Unidad de Aprendizaje:

Sistemas Operativos

Práctica

Inicio de sesión e interfaces basadas en texto

Equipo:
Alvarez Garcia Martin
Hernandez Castillo Emir Adrian
Joaquín Rojas González
Nava Alvarado Donovan Emilio

Grupo:
4CV4

Profesor:
José Alfredo Jiménez Benítez

Ciudad de México, 11 de octubre de 2026

Resumen

El presente proyecto consiste en el desarrollo de un sistema de inicio de sesión e interfaces basadas en texto, implementado en lenguaje C mediante la biblioteca `ncurses`. El sistema, denominado JDM E-TICKETS, permite realizar el registro de usuarios y la autenticación mediante una interfaz interactiva en terminal. Su diseño integra mecanismos de validación de datos, procesamiento criptográfico de contraseñas y almacenamiento local de información, con el propósito de establecer un flujo de acceso que contemple tanto la interacción con el usuario como el manejo de sus credenciales.

La interfaz permite desplazarse entre las opciones disponibles mediante el teclado, utilizando las teclas de dirección, `ENTER` y `ESC`. A través de los formularios se capturan los datos necesarios para crear una cuenta o iniciar sesión. Durante el registro se verifican condiciones como la longitud mínima del nombre de usuario, la estructura de la dirección de correo electrónico, la disponibilidad de ambos identificadores, la complejidad de la contraseña y la coincidencia entre la contraseña y su confirmación. Estas validaciones permiten controlar la información antes de completar el registro y proporcionan una respuesta cuando los datos ingresados no satisfacen los requisitos establecidos.

En materia de seguridad, el sistema utiliza la biblioteca `libsodium` para generar y verificar hashes de contraseñas mediante el algoritmo Argon2id. De esta manera, las contraseñas originales no necesitan almacenarse directamente en el archivo de usuarios. La información persistente se conserva localmente en `usuarios.txt`, donde cada registro contiene el nombre de usuario, el correo electrónico y el hash correspondiente. Este mecanismo permite mantener las cuentas disponibles entre distintas ejecuciones de la aplicación, sin depender de un sistema gestor de bases de datos o de un servicio remoto.

La organización del código fuente separa el flujo principal, las funciones de autenticación y los componentes de la interfaz, facilitando la identificación de las responsabilidades de cada módulo. Asimismo, la compilación se administra mediante un archivo `Makefile`, que define las opciones y las bibliotecas necesarias para generar el ejecutable.

El proyecto integra conceptos de programación en C, manejo de terminal, validación de entradas, persistencia en archivos y protección criptográfica de credenciales. Su implementación permite estudiar cómo interactúan estos elementos en una aplicación de autenticación de alcance local y cómo una interfaz textual puede proporcionar navegación, mensajes de estado y formularios estructurados sin recurrir a un entorno gráfico.

Palabras clave: sistemas operativos, lenguaje C, `ncurses`, autenticación, Argon2id, persistencia local y validación de datos.

Contenido

Resumen **1**

1. Introducción **3**

2. Marco Teórico **4**

2.1. Teoría sobre la creación de un inicio de sesión 4

2.1.1. Identificación y autenticación de usuarios 4

2.1.2. El flujo de inicio de sesión en sistemas UNIX 4

2.1.3. Archivos de control y protección de credenciales (`/etc/passwd` y `/etc/shadow`) . 5

2.2. Algoritmos de cifrado 5

2.3. Biblioteca ncurses 5

3. Desarrollo **6**

3.1. Organización del código fuente 6

3.2. Compilación del programa 6

3.3. Implementación de la interfaz con ncurses 7

3.4. Registro de usuarios y validación de datos 8

3.5. Protección de contraseñas y autenticación 10

3.6. Registro y persistencia en archivos 12

3.7. Pruebas funcionales 13

4. Conclusiones **15**

4.1. Alvarez Garcia Martin 15

4.2. Hernandez Castillo Emir Adrian 16

4.3. Joaquín Rojas González 17

4.4. Nava Alvarado Donovan Emilio 18

Referencias **19**

5. Referencias **19**

Anexos **20**

1 Introducción

Los sistemas de inicio de sesión constituyen un mecanismo fundamental para controlar el acceso a las aplicaciones que administran información asociada con distintos usuarios. Su funcionamiento requiere identificar a quien intenta ingresar, verificar sus credenciales y gestionar adecuadamente los datos utilizados durante el registro y la autenticación. Además de comprobar la correspondencia entre un nombre de usuario y una contraseña, resulta necesario contemplar la validación de las entradas, el almacenamiento persistente de las cuentas y la presentación de mensajes que permitan comprender el resultado de cada operación.

En el contexto de los sistemas operativos, el desarrollo de una aplicación de estas características permite estudiar la interacción entre un programa y el entorno en el que se ejecuta. El manejo del teclado, la presentación de información en una terminal, la lectura y escritura de archivos y el uso de bibliotecas externas forman parte de los elementos que intervienen en su funcionamiento. Estos componentes adquieren especial relevancia cuando la aplicación debe mantener una interacción organizada sin utilizar ventanas, botones gráficos ni otros elementos propios de una interfaz visual tradicional.

A partir de este contexto, se desarrolló JDM E-TICKETS, un sistema de registro e inicio de sesión implementado en lenguaje C. La aplicación utiliza la biblioteca `ncurses` para construir una interfaz basada en texto que permite navegar por las opciones y completar formularios mediante el teclado. Su funcionamiento contempla dos operaciones principales: la creación de cuentas nuevas y la autenticación de usuarios registrados. Para el registro, se establecen condiciones sobre el nombre de usuario, la dirección de correo electrónico y la contraseña, mientras que el inicio de sesión se basa en la existencia de la cuenta y la verificación de la contraseña proporcionada.

Otro aspecto relevante del proyecto es el tratamiento de las credenciales. En lugar de conservar las contraseñas originales, la aplicación emplea `libsodium` y el algoritmo Argon2id para generar hashes que posteriormente se utilizan durante la autenticación. Los registros se almacenan en un archivo local denominado `usuarios.txt`, lo que permite conservar la información entre ejecuciones sin incorporar una base de datos ni depender de un servidor externo. Esta decisión delimita el alcance de la solución y permite concentrar su implementación en el procesamiento de datos, la interacción con el usuario y las operaciones de archivos.

El objetivo principal del proyecto es implementar un mecanismo funcional de registro y autenticación que integre una interfaz textual, validaciones de entrada, verificación criptográfica de contraseñas y persistencia local. Para ello, el código se organiza en módulos que separan la lógica principal, las funciones de autenticación y los componentes visuales, mientras que el proceso de compilación se administra mediante un archivo `Makefile`.

El reporte presenta los fundamentos relacionados con el funcionamiento del sistema y describe las decisiones tomadas durante su implementación. Asimismo, aborda la estructura del código fuente, la compilación, la construcción de la interfaz, las reglas de validación, el tratamiento de las contraseñas y el almacenamiento de los usuarios. Finalmente, se incluyen las conclusiones correspondientes y las referencias utilizadas para contextualizar el desarrollo. De esta manera, se expone la relación entre los conceptos estudiados y su aplicación en un programa de autenticación ejecutado desde una terminal de Linux.

2 Marco Teórico

2.1 Teoría sobre la creación de un inicio de sesión

La autenticación de usuarios es el mecanismo fundamental que permite a un sistema operativo validar la identidad de los sujetos que solicitan acceso a sus recursos. De acuerdo con Silberschatz, Galvin y Gagne en Fundamentos de Sistemas Operativos, es indispensable establecer una distinción clara entre la identificación y la autenticación [2]. La identificación ocurre cuando un usuario declara su identidad mediante una credencial o nombre de cuenta. Por su parte, la autenticación comprende la comprobación explícita de dicha declaración. Las técnicas de autenticación se dividen fundamentalmente en tres categorías: información que el usuario conoce, objetos que el usuario posee e indicadores biométricos que identifican sus características físicas o conductuales.

En el contexto de la administración de accesos en entornos tipo UNIX, Francisco Manuel Márquez García describe en Unix Programación Avanzada la secuencia técnica de llamadas al sistema encargadas del flujo de inicio de sesión [3]. La arquitectura de estos sistemas delega la gestión de las líneas de terminal al proceso inicializador del sistema, el cual invoca rutinas especializadas para configurar la línea de comunicación y solicitar el identificador del usuario. Posteriormente, se transfiere el control al ejecutable de inicio de sesión, el cual desactiva el eco en la terminal, solicita la clave de acceso y consulta los archivos de seguridad del sistema antes de inicializar el entorno mediante el establecimiento de identificadores de usuario y la ejecución del intérprete de comandos.

La arquitectura de protección de credenciales en el sistema operativo exige una separación rigurosa en el almacenamiento de datos. Como exponen Andrew S. Tanenbaum y Herbert Bos en Sistemas Operativos Modernos, las contraseñas nunca deben almacenarse en texto plano [1]. Históricamente, la información técnica de los usuarios se concentraba en el archivo del sistema de acceso público; sin embargo, los sistemas modernos implementan mecanismos de protección que restringen los permisos de lectura únicamente al usuario con privilegios de administrador, albergando los resúmenes cifrados de las contraseñas junto con los parámetros de expiración de las cuentas.

2.1.1 Identificación y autenticación de usuarios

El proceso de identificación constituye el primer paso dentro del control de acceso en un sistema operativo multiusuario. En esta etapa, el usuario proporciona un identificador único, como un nombre de usuario o un número de cuenta, para declararse ante el sistema. No obstante, la sola declaración de la identidad es insuficiente para otorgar privilegios sobre los recursos del sistema operativo, por lo que se requiere un procedimiento riguroso de autenticación. Silberschatz, Galvin y Gagne destacan que la autenticación verifica la validez de la afirmación presentada [2]. La forma más extendida de autenticación se basa en secretos compartidos, como las contraseñas numéricas o alfanuméricas, donde el usuario demuestra su conocimiento sobre un dato privado previamente registrado. En entornos que requieren mayor grado de seguridad, los sistemas operativos integran múltiples factores combinados para reducir las vulnerabilidades derivadas del robo o la adivinación de claves de acceso.

2.1.2 El flujo de inicio de sesión en sistemas UNIX

En los sistemas operativos basados en la filosofía UNIX, el inicio de sesión es administrado mediante una jerarquía de procesos encadenados. Márquez García explica que el proceso raíz o inicializador del sistema genera procesos dedicados a supervisar cada terminal o línea de entrada disponible [3]. Cuando un usuario interactúa con la terminal, la rutina `getty` establece la velocidad de transmisión, los caracteres de control de la línea y despliega el mensaje de bienvenida solicitando el nombre de usuario. Tras la recepción del nombre de

usuario, dicho proceso sustituye su imagen en memoria mediante la llamada al sistema `execve` para invocar al programa `login`. El ejecutable `login` se encarga de cambiar la configuración del controlador de la terminal para deshabilitar el eco del teclado, solicita la contraseña, aplica el algoritmo de verificación y, en caso de autenticación exitosa, asigna los identificadores de usuario y grupo correspondientes mediante llamadas al sistema como `setuid` y `setgid`, finalizando con la ejecución del shell configurado para el usuario.

2.1.3 Archivos de control y protección de credenciales (`/etc/passwd` y `/etc/shadow`)

La estructura de almacenamiento de credenciales en sistemas UNIX evolucionó debido a las limitaciones de seguridad del diseño original. Tanenbaum y Bos detallan que tradicionalmente el archivo `/etc/passwd` guardaba todos los datos de la cuenta de usuario, incluyendo el nombre, la contraseña cifrada, el identificador de usuario, el identificador de grupo predeterminado, el directorio de trabajo y el intérprete de comandos asignado [1]. Dado que numerosos programas legítimos del sistema necesitan consultar la información de los usuarios en `/etc/passwd`, dicho archivo debe conservar permisos de lectura global para todos los procesos del sistema. Esta visibilidad pública permitía que usuarios malintencionados copiaran el archivo y aplicaran técnicas de fuerza bruta o diccionarios fuera de línea sobre las contraseñas cifradas. Para erradicar este riesgo, se introdujo la técnica del archivo en sombra o `/etc/shadow`, el cual separa las contraseñas cifradas y las traslada a un archivo protegido accesible únicamente por el usuario administrador. En `/etc/passwd` el campo de la contraseña se sustituye por una marca fija, mientras que `/etc/shadow` resguarda los datos sensibles, la sal de cifrado y los parámetros de expiración de la clave.

2.2 Algoritmos de cifrado

El almacenamiento de contraseñas en texto plano representa una vulnerabilidad crítica. Para mitigar este riesgo, los sistemas implementan algoritmos de cifrado o funciones de un solo sentido (hashing). En lenguaje C, es común utilizar funciones matemáticas o bibliotecas nativas como `crypt()` (basada en DES, MD5 o SHA) que transforman la contraseña original en una cadena de caracteres ilegible. Durante el inicio de sesión, el sistema cifra la contraseña ingresada y compara el resultado con el hash guardado; si ambos coinciden, se otorga el acceso sin que el sistema necesite conocer la clave original.

2.3 Biblioteca ncurses

`ncurses` (new curses) es una biblioteca de programación que proporciona una API para construir interfaces de usuario basadas en texto de manera independiente de la terminal. Permite controlar la posición del cursor, crear ventanas, utilizar colores y leer el teclado carácter por carácter. Funciones como `noecho()` son esenciales en los inicios de sesión, ya que desactivan el eco visual de la terminal, permitiendo que el usuario escriba su contraseña sin que los caracteres sean visibles en la pantalla.

3 Desarrollo

El desarrollo del sistema de inicio de sesión se realizó en lenguaje C, utilizando la biblioteca `ncurses` para construir una interfaz de usuario basada en texto y la biblioteca `libsodium` para el procesamiento de contraseñas. El sistema permite registrar cuentas, autenticar usuarios y conservar la información necesaria entre ejecuciones. Para organizar estas operaciones, la implementación se dividió en módulos que separan la lógica de autenticación, los componentes visuales y el flujo principal, lo que permite distinguir las responsabilidades de cada parte del programa.

3.1 Organización del código fuente

El proyecto se organiza en varios archivos fuente y de cabecera, cada uno con una responsabilidad definida dentro de la aplicación. El archivo `main.c` contiene el flujo principal y coordina las operaciones de inicio de sesión y registro, mientras que `auth.c` implementa las funciones relacionadas con la autenticación, la validación de credenciales y el manejo de la información persistente. Por su parte, `auth.h` declara las interfaces del módulo de autenticación, y los archivos `ui.c` y `ui.h` contienen las funciones y declaraciones relacionadas con la presentación de formularios, mensajes y opciones mediante `ncurses`. Finalmente, el archivo `Makefile` reúne las instrucciones necesarias para compilar y enlazar el programa.

La separación entre autenticación e interfaz mantiene diferenciadas las operaciones que procesan las credenciales y aquellas que presentan los formularios y mensajes en pantalla. El archivo principal coordina ambos módulos para establecer el flujo de navegación de la aplicación, mientras que las interfaces declaradas en los archivos de cabecera permiten conectar las funciones entre sí de manera organizada.

3.2 Compilación del programa

La compilación se administra mediante un archivo `Makefile`, que reúne las opciones del compilador, los archivos fuente y las bibliotecas requeridas por el proyecto [7]. Para generar el ejecutable se utiliza GCC con el estándar C11 y opciones de advertencia como `-Wall`, `-Wextra` y `-Wpedantic`, que ayudan a identificar posibles errores y construcciones que no se ajustan al estándar seleccionado. La opción `-O2` aplica optimizaciones al código generado.

El programa requiere las bibliotecas `ncurses`, `tinfo` y `sodium`. Las bibliotecas relacionadas con `ncurses` proporcionan las funciones necesarias para trabajar con la terminal, mientras que `libsodium` incorpora las operaciones criptográficas empleadas para procesar las contraseñas. Estas dependencias deben estar disponibles en el entorno para que el proceso de compilación pueda completarse correctamente.

La compilación del proyecto se inicia desde el directorio que contiene el archivo `Makefile` mediante el comando siguiente:

```
make
```

Cuando la compilación finaliza correctamente, se genera el ejecutable `acceso`, que puede iniciarse desde el mismo directorio mediante el comando siguiente:

```
./acceso
```

La Figura 1 muestra la salida de la terminal durante la compilación del proyecto y sirve como evidencia visual del proceso de construcción del ejecutable.

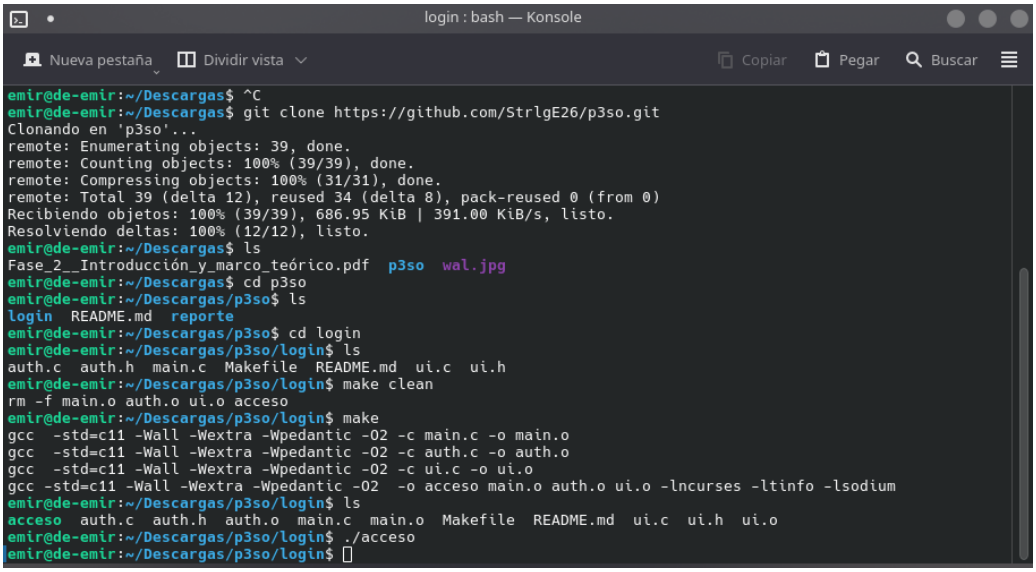


Figura 1: Compilación del sistema mediante el archivo `Makefile`.

3.3 Implementación de la interfaz con ncurses

La interfaz de usuario se desarrolló mediante la biblioteca `ncurses`, que permite controlar la presentación de texto, la posición del cursor y la lectura de entradas del teclado dentro de una terminal [4]. Su utilización hace posible organizar los formularios y los mensajes de la aplicación sin recurrir a una interfaz gráfica tradicional, manteniendo la interacción dentro del entorno de consola.

La pantalla principal presenta las opciones relacionadas con el acceso y el registro de cuentas, y los campos de entrada se representan visualmente como cuadros de texto. Las opciones seleccionadas se distinguen mediante resaltados de color, mientras que la combinación de fondo oscuro y texto claro mantiene una presentación uniforme en las distintas pantallas. La Figura 2 muestra la pantalla inicial de JDM E-TICKETS y la distribución de las opciones disponibles para el usuario.

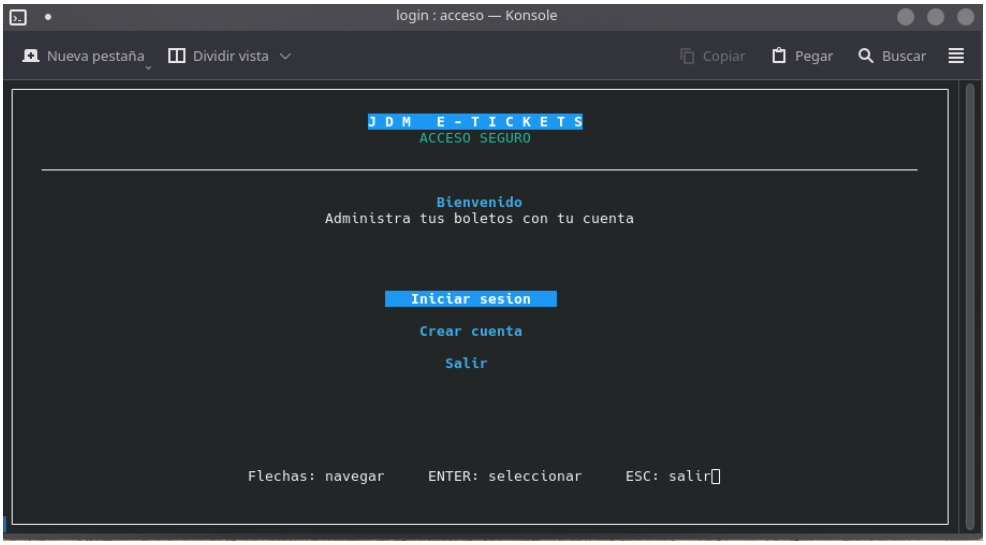


Figura 2: Menú principal de JDM E-TICKETS.

La navegación se realiza mediante las teclas de dirección, `ENTER` y `ESC`. Las teclas de dirección permiten desplazarse entre las opciones disponibles, `ENTER` confirma la selección o permite continuar con una operación y `ESC` permite regresar o cancelar la acción correspondiente de acuerdo con el contexto de la interfaz. La configuración de `ncurses` incluye funciones como `noecho()`, que evita que la entrada se muestre automáticamente en pantalla, y `keypad()`, que permite interpretar teclas especiales según la configuración del terminal [4]. Estas funciones centralizan la interacción con el teclado y contribuyen a mantener un comportamiento consistente entre las pantallas de autenticación y registro.

3.4 Registro de usuarios y validación de datos

El registro permite crear nuevas cuentas mediante un nombre de usuario, una dirección de correo electrónico y una contraseña con su respectiva confirmación. Antes de almacenar una cuenta, el sistema verifica que los datos ingresados cumplan con las reglas de validación definidas para el proyecto, con el propósito de evitar registros incompletos, duplicados o incompatibles con los requisitos establecidos.

El nombre de usuario debe contener al menos cinco caracteres y no debe encontrarse registrado previamente, lo que evita el uso de identificadores demasiado cortos y permite mantener su unicidad. De manera similar, la dirección de correo electrónico debe presentar una estructura válida y no estar asociada a otra cuenta. Esta comprobación revisa la sintaxis de la dirección proporcionada, pero no confirma que el buzón exista ni que el usuario tenga acceso a él.

La contraseña debe cumplir con los requisitos de complejidad establecidos, que incluyen letras mayúsculas, letras minúsculas, números y símbolos. Además, el sistema solicita una segunda entrada para confirmar la contraseña y ambas entradas deben coincidir para que el registro pueda continuar. Cuando un campo no cumple con las reglas correspondientes, el flujo de registro permite corregir la entrada inválida sin perder los valores válidos que ya se habían introducido en los demás campos.

Las Figuras 3, 4, 5 y 6 muestran ejemplos de las validaciones aplicadas durante el registro: el rechazo de un nombre de usuario no válido, la detección de una dirección de correo incorrecta, una contraseña que no cumple los requisitos de complejidad y una confirmación que no coincide. La Figura 7 presenta el mensaje que aparece cuando una cuenta se registra correctamente. En conjunto, estas evidencias permiten relacionar las reglas de validación con las respuestas que presenta la interfaz.

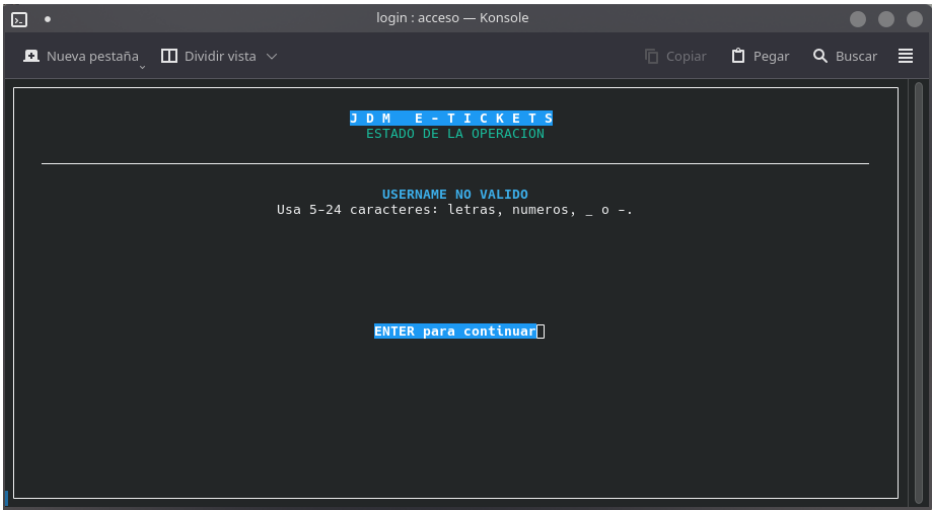


Figura 3: Validación de un nombre de usuario que no cumple con las condiciones de registro.

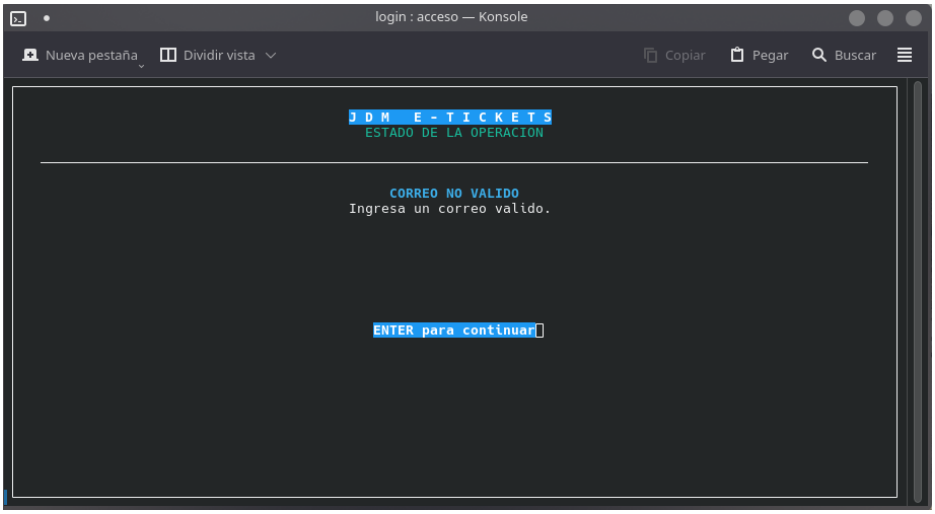


Figura 4: Mensaje de validación al ingresar una dirección de correo electrónico con estructura incorrecta.

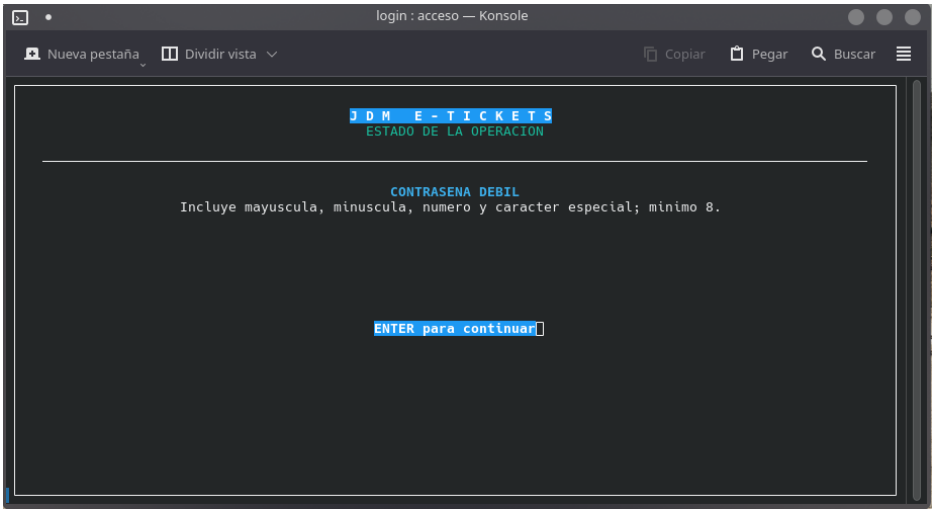


Figura 5: Aviso presentado cuando la contraseña no satisface los requisitos de complejidad.

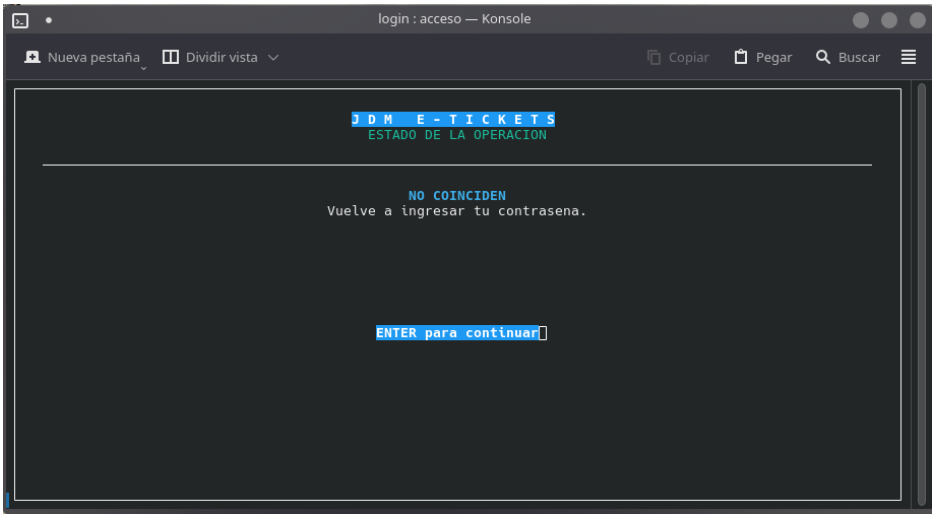


Figura 6: Mensaje presentado cuando la contraseña y su confirmación son diferentes.

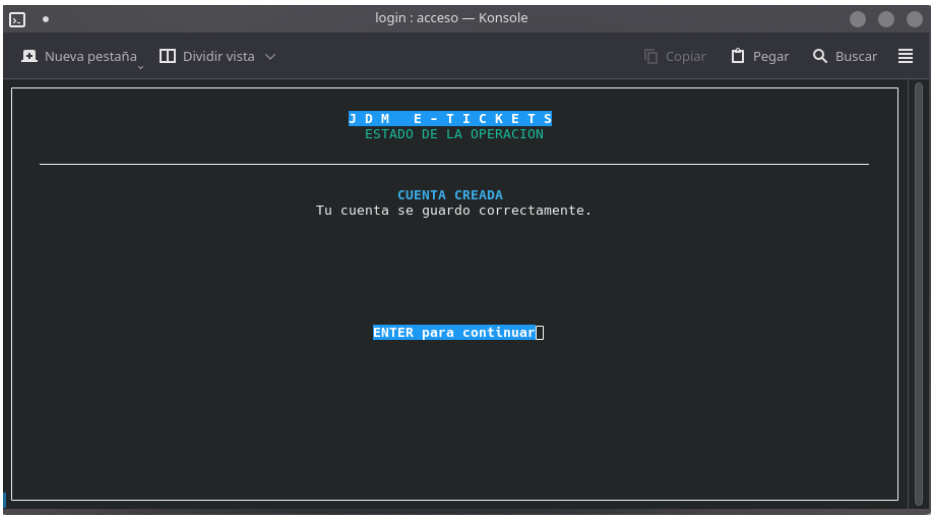


Figura 7: Confirmación visual del registro exitoso de una cuenta.

3.5 Protección de contraseñas y autenticación

El tratamiento de las contraseñas es una parte esencial del sistema, ya que estos datos se utilizan para comprobar la identidad de los usuarios. Por esta razón, la aplicación emplea las funciones criptográficas de `libsodium` para generar y verificar hashes de contraseñas mediante el algoritmo Argon2id, evitando que la contraseña original tenga que conservarse directamente en el archivo de usuarios [6].

Durante el registro, la función `crypto_pwhash_str()` genera una representación criptográfica destinada al almacenamiento e incorpora los parámetros necesarios para verificar la contraseña posteriormente. Durante el inicio de sesión, el sistema busca el registro correspondiente al nombre de usuario y utiliza `crypto_pwhash_str_verify()` para comprobar la contraseña ingresada frente al hash almacenado. La autenticación se considera correcta cuando la cuenta existe y la verificación criptográfica confirma que la contraseña proporcionada es válida.

El comportamiento de la interfaz depende del resultado de esta validación. Cuando la contraseña es incorrecta, el campo correspondiente se limpia para permitir una nueva entrada, mientras que el nombre de

usuario se conserva; si el nombre de usuario no existe, se limpian ambos campos para que el siguiente intento comience con los datos de acceso vacíos. Argon2id se utiliza para la verificación de contraseñas mediante hashes y no equivale al cifrado reversible, ya que el sistema no necesita recuperar la contraseña original.

Adicionalmente, las funciones de limpieza de memoria de `libsodium`, como `sodium_memzero()`, permiten borrar explícitamente determinados datos sensibles de los búferes utilizados por el programa cuando corresponde. Esta medida complementa el manejo de credenciales al reducir el tiempo durante el cual ciertos datos permanecen en memoria. Las Figuras 8, 9, 10 y 11 ilustran los principales estados del inicio de sesión: el formulario de entrada, el rechazo de una contraseña incorrecta, la respuesta ante un usuario inexistente y la confirmación de una autenticación correcta.

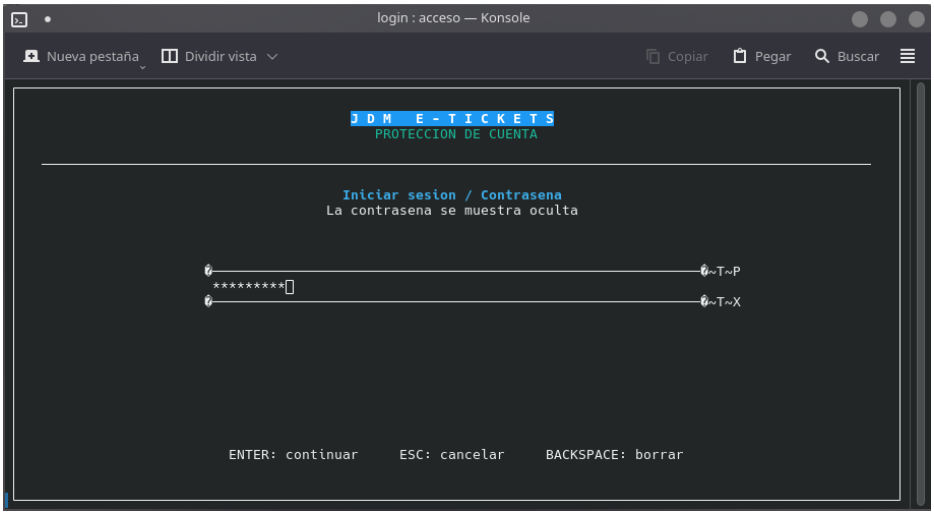


Figura 8: Formulario de inicio de sesión con el campo de contraseña.

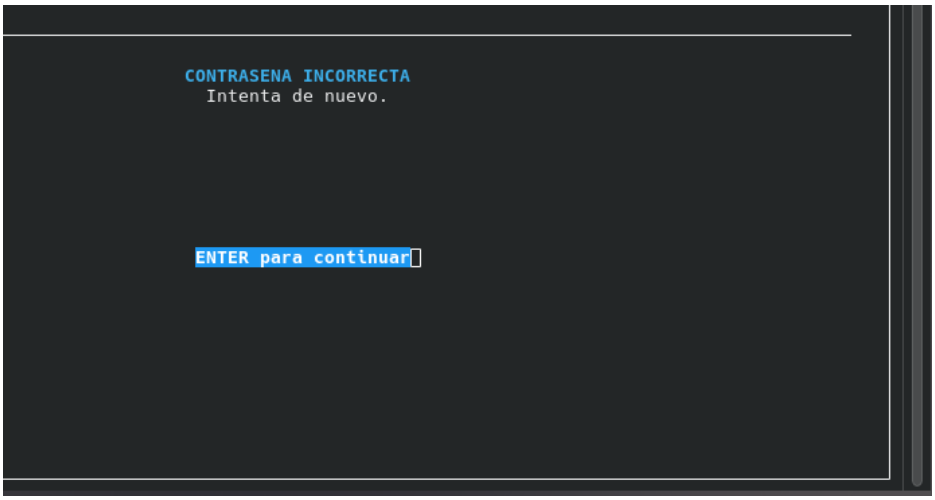


Figura 9: Respuesta de la aplicación ante una contraseña incorrecta.

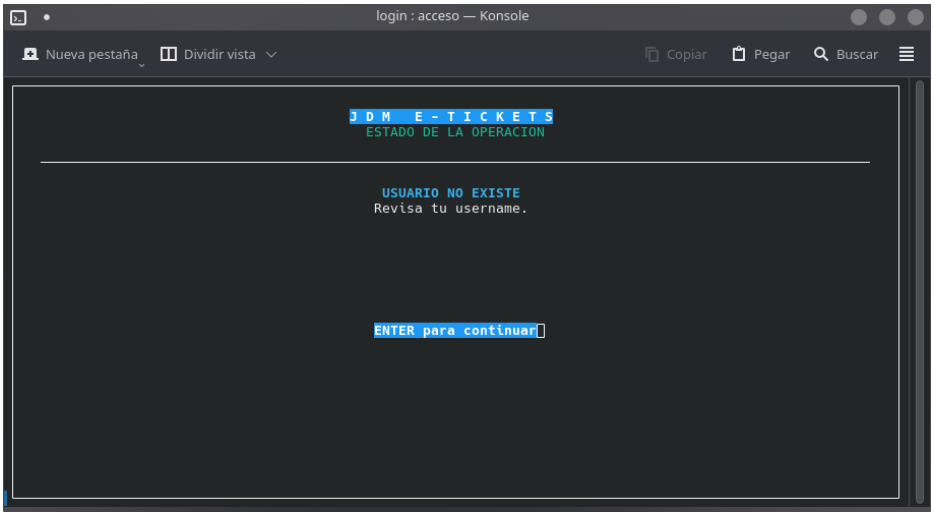


Figura 10: Respuesta de la aplicación cuando el nombre de usuario no está registrado.

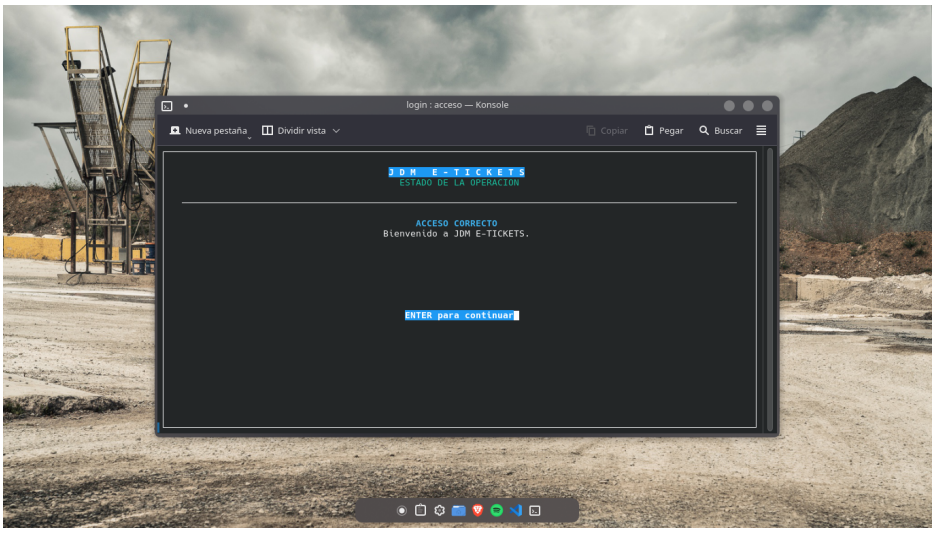


Figura 11: Confirmación visual de un inicio de sesión correcto.

3.6 Registro y persistencia en archivos

La información de las cuentas se conserva en un archivo local denominado `usuarios.txt`, lo que permite mantener los registros entre ejecuciones sin requerir un sistema gestor de bases de datos ni una conexión a un servidor remoto. El uso de archivos como mecanismo de persistencia requiere considerar las operaciones de lectura y escritura, así como los permisos de acceso y la protección de la información almacenada [**?**, 4, 5]. Los datos se almacenan en registros delimitados por dos puntos, con el nombre de usuario, el correo electrónico y el hash de la contraseña, de acuerdo con la estructura siguiente:

```
usuario:correo:hash
```

En esta estructura, el primer campo corresponde al nombre de usuario, el segundo contiene la dirección de correo electrónico y el tercero almacena el hash generado para la contraseña. Por lo tanto, la contraseña original no forma parte del registro persistente. La lectura del archivo permite localizar las cuentas existentes y

recuperar la información necesaria para comprobar la unicidad del nombre de usuario y del correo electrónico, así como para realizar la autenticación.

Cuando se registra una nueva cuenta, la información correspondiente se incorpora al archivo para que permanezca disponible después de cerrar y volver a ejecutar el programa. Este enfoque evita depender de servicios externos para las operaciones básicas del sistema, aunque la disponibilidad y la integridad de los registros siguen dependiendo de los permisos, el estado y la administración del archivo en el equipo donde se ejecuta la aplicación. El almacenamiento de hashes reduce la exposición directa de las credenciales, pero no elimina la necesidad de proteger el archivo y el entorno de ejecución. La Figura 12 muestra un ejemplo de la estructura de los datos persistidos localmente y permite distinguir los campos que componen cada registro.

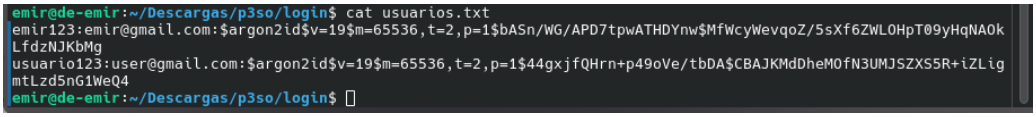


Figura 12: Estructura de los registros almacenados localmente en el archivo de usuarios.

3.7 Pruebas funcionales

Las pruebas funcionales permiten evaluar el comportamiento del sistema durante el registro de cuentas, la autenticación de usuarios y la conservación de los registros. Los escenarios considerados incluyen operaciones exitosas y situaciones en las que los datos ingresados no cumplen con las reglas de validación. La Tabla 1 reúne doce escenarios representativos y describe tanto las condiciones de entrada como el resultado esperado en cada caso. Esta organización permite relacionar cada regla de negocio con el comportamiento que debe observarse durante la ejecución del programa.

Tabla 1: Casos de prueba funcionales y resultados esperados.

N.º	Escenario	Entrada o condición	Resultado esperado
1	Registro correcto	Datos válidos y cuenta nueva	La cuenta se registra y queda almacenada para futuras ejecuciones.
2	Nombre de usuario corto	Menos de cinco caracteres	El registro solicita corregir el nombre de usuario.
3	Usuario duplicado	Nombre ya registrado	El sistema rechaza el registro duplicado.
4	Correo inválido	Dirección con estructura incorrecta	El sistema solicita corregir la dirección de correo electrónico.
5	Correo duplicado	Dirección ya registrada	El sistema rechaza el registro duplicado.
6	Contraseña débil	No incluye todos los tipos de caracteres requeridos	El sistema solicita una contraseña que cumpla los requisitos de complejidad.
7	Confirmación incorrecta	Contraseña y confirmación diferentes	El registro no continúa hasta que ambas entradas coincidan.
8	Inicio de sesión correcto	Usuario existente y contraseña válida	El sistema permite completar la autenticación.
9	Contraseña incorrecta	Usuario existente y contraseña errónea	El sistema rechaza el acceso, limpia la contraseña y conserva el nombre de usuario.
10	Usuario inexistente	Nombre de usuario no registrado	El sistema rechaza el acceso y limpia ambos campos.
11	Persistencia de datos	Cuenta registrada y aplicación ejecutada nuevamente	La cuenta continúa disponible para la autenticación.
12	Navegación por teclado	Uso de flechas, `ENTER` y `ESC`	La interfaz permite navegar, seleccionar opciones y regresar o cancelar según el contexto.

Los escenarios anteriores abarcan las operaciones principales del sistema y permiten distinguir los resultados de un registro válido de aquellos producidos por entradas incorrectas. La comprobación de estos comportamientos permite evaluar la correspondencia entre las reglas de negocio, la lógica de autenticación y la interacción presentada en la terminal.

Los resultados observados durante las pruebas deben interpretarse de acuerdo con las condiciones de cada escenario. En particular, el registro correcto debe conservar la cuenta en el archivo local, mientras que los datos inválidos deben impedir la creación de registros que incumplan las reglas establecidas. De forma equivalente, el inicio de sesión debe permitir el acceso únicamente cuando la cuenta existe y la contraseña supera la verificación criptográfica.

La persistencia se evalúa mediante la disponibilidad de los registros después de finalizar y volver a iniciar la aplicación, mientras que las pruebas de navegación permiten comprobar que los controles del teclado mantienen el comportamiento previsto en las distintas pantallas. En conjunto, estos escenarios describen los aspectos funcionales esenciales de la aplicación, sin sustituir las comprobaciones específicas de seguridad, concurrencia o resistencia ante modificaciones externas del archivo de usuarios.

4 Conclusiones

4.1 Alvarez Garcia Martin

4.2 Hernandez Castillo Emir Adrian

La realización de esta práctica me permitió comprender de manera más clara cómo se relacionan la programación en C, el manejo de archivos y la interacción con el usuario dentro de una aplicación ejecutada en Linux. Uno de los principales aprendizajes fue identificar que el desarrollo de un sistema de inicio de sesión no consiste únicamente en solicitar un nombre de usuario y una contraseña, sino que también requiere validar los datos, administrar las credenciales y mantener la información disponible entre distintas ejecuciones.

Uno de los aspectos que considero más relevantes fue la separación del código en módulos. Al organizar el flujo principal, la lógica de autenticación y los elementos de la interfaz en archivos independientes, comprendí la importancia de establecer responsabilidades específicas dentro del programa. Esta organización facilita la comprensión del funcionamiento del sistema y permite realizar modificaciones sin concentrar toda la lógica en un solo archivo.

El uso de `ncurses` también me permitió comprender mejor la interacción entre una aplicación y la terminal. Mediante el manejo de las teclas de dirección, `ENTER` y `ESC`, fue posible construir una interfaz que permite desplazarse entre las opciones y completar los formularios sin recurrir a ventanas gráficas. Además, el tratamiento de los mensajes de error y de los campos de entrada me ayudó a identificar la importancia de proporcionar respuestas claras cuando el usuario introduce información incorrecta.

En cuanto a la seguridad, comprendí que almacenar las contraseñas directamente en un archivo representa una práctica inadecuada para el manejo de credenciales. Por esta razón, el uso de `libsodium` y del algoritmo Argon2id permitió incorporar un mecanismo de generación y verificación de hashes sin necesidad de conservar las contraseñas originales. Este aspecto me permitió distinguir entre el cifrado reversible y el procesamiento de contraseñas mediante hashes, así como reconocer la importancia de utilizar funciones criptográficas especializadas en lugar de implementar estos procedimientos desde cero.

Por otra parte, el almacenamiento de las cuentas en el archivo `usuarios.txt` me permitió comprender que la persistencia de la información depende de las operaciones de lectura y escritura realizadas por el propio programa. Al no utilizar un gestor de bases de datos, el sistema debe encargarse de organizar los registros y recuperar los datos necesarios durante la autenticación. También identifiqué que mantener la información en un archivo local implica considerar su integridad, sus permisos y la protección de los datos almacenados.

Otro aprendizaje importante fue la necesidad de validar las entradas antes de completar el registro. Las condiciones relacionadas con el nombre de usuario, el correo electrónico, la complejidad de la contraseña y su confirmación permiten controlar los datos que recibe la aplicación y evitar registros que no cumplan con las reglas establecidas. Asimismo, el tratamiento diferenciado de los errores de autenticación demuestra que el comportamiento de la interfaz debe corresponder con el resultado de las comprobaciones realizadas por el programa.

Finalmente, el uso de un archivo `Makefile` me permitió reconocer la importancia de organizar el proceso de compilación y especificar las dependencias necesarias para generar el ejecutable. En conclusión, esta práctica me permitió relacionar los conceptos de programación en C, manejo de archivos, entrada y salida mediante terminal y protección de credenciales con su aplicación en un sistema funcional. Considero que el principal aprendizaje fue comprender que la confiabilidad de una aplicación de autenticación depende de la integración adecuada de sus componentes y no solamente de que las operaciones principales puedan ejecutarse. Estos conocimientos constituyen una base para continuar desarrollando aplicaciones en Linux con una estructura más organizada y un manejo más cuidadoso de la información.

4.3 Joaquín Rojas González

4.4 Nava Alvarado Donovan Emilio

5 Referencias

Referencias

[1] A. S. Tanenbaum y H. Bos, *Sistemas Operativos Modernos*, 5.a ed. México: Pearson Educación, 2024.

[2] A. Silberschatz, P. B. Galvin y G. Gagne, *Fundamentos de Sistemas Operativos*, 7.a ed. México: McGraw Hill Education, 2006.

[3] F. M. Márquez García, *UNIX Programación avanzada*, 3.a ed. ampliada y actualizada. Madrid: Ra-Ma, 2004.

[4] W. R. Stevens y S. A. Rago, *Advanced Programming in the UNIX Environment*, 3.a ed. Addison-Wesley Professional, 2013.

[5] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA: No Starch Press, 2010.

[6] Libsodium. *Password Hashing*. Documentación oficial. Disponible en: `https://doc.libsodium.org/password_hashing`.

[7] GNU Project. *GNU Make Manual*. Disponible en: `https://www.gnu.org/software/make/manual/make.html`.

Anexos

En esta sección se incluye el repositorio oficial donde se encuentra el código fuente en lenguaje C, los archivos de cabecera y el archivo Makefile necesarios para la compilación del sistema:

`https://github.com/StrlgE26/p3so.git`